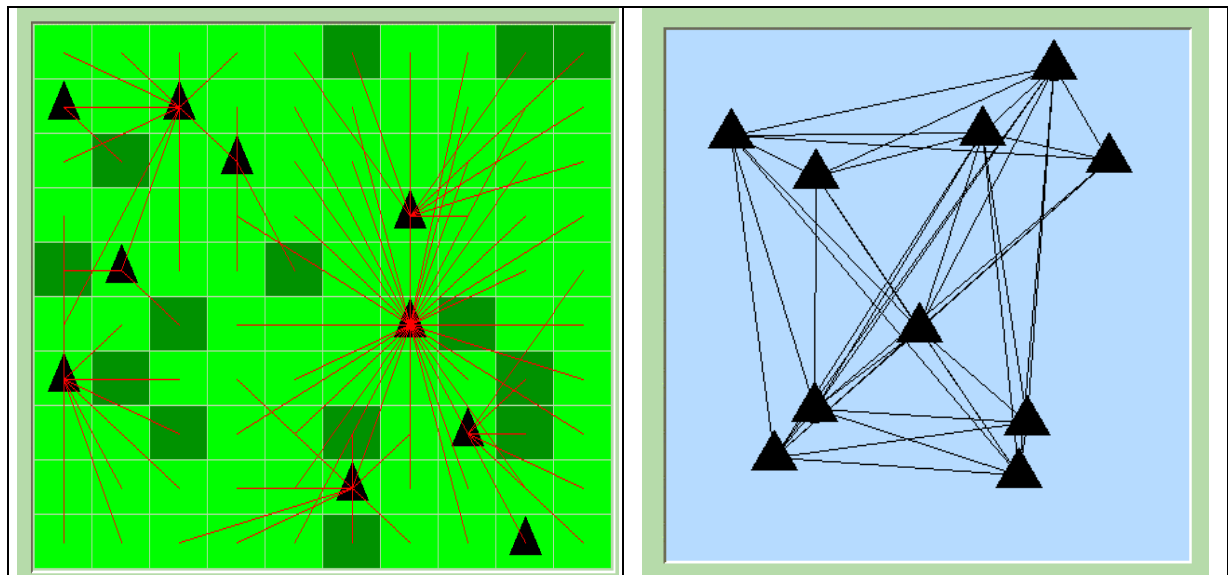


CIRAD
Department of Territories, Environment and People
Land and Resources Programme

CORMAS
Common-pool Resources
and Multi-Agents Systems

User's Guide



April 2001

1	GETTING STARTED.....	3
1.1	INSTALLING VISUALWORKS.....	3
1.1.1	Downloading VisualWorks 3.0 Non-Commercial.....	3
1.1.2	Unzipping and Moving VisualWorks 3.0 Non-Commercial Files.....	3
1.2	INSTALLING CORMAS.....	4
1.2.1	Downloading Cormas.....	4
1.2.2	Unzipping and Moving the Cormas Files.....	4
1.2.3	Copying the basic VisualWorks Image.....	4
1.2.4	Opening VisualWorks.....	4
1.2.5	Adujst the VisualWorks Home Directory.....	4
1.2.6	Adding Required Parcels.....	5
1.2.7	Adding Cormas Specific Tools to the VisualWorks Launcher.....	5
1.2.8	Adjust the VisualWorks Settings.....	5
1.2.9	Save the VisualWorks Environment set up for Cormas.....	6
1.3	OPENING CORMAS.....	6
2	BUILDING A MODEL WITH CORMAS.....	8
2.1	DEFINITION OF THE ENTITIES.....	8
2.1.1	How to Create, Modify or Delete Entities.....	8
2.1.2	The Cormas Classes' Hierarchy and the Root Class <i>Entity</i>	10
2.1.3	Spatial Entities.....	12
2.1.4	Passive Entities.....	16
2.1.4.1	The passive objects of Cormas.....	16
2.1.4.2	How to create a message ?.....	16
2.1.5	Agents.....	18
2.1.5.1	AgentLocation.....	18
2.1.5.2	AgentComm.....	18
2.1.5.3	Group.....	20
2.2	CONTROLLING THE EVOLUTION OF THE MODEL.....	21
2.2.1	Initialisation of a model.....	22
2.2.2	Control of the model.....	22
2.3	HOW TO OBSERVE THE MODEL.....	23
2.3.1	Observation of the space.....	23
2.3.1.1	Definition of a point of view.....	24
2.3.1.2	Definition of a symbol.....	24
2.3.1.3	Definition of the shape, size and color.....	24
2.3.2	Observation of Communication.....	25
2.3.3	Charts.....	25
3	OPENING, IMPORTING, EXPORTING MODELS.....	27
3.1	MODELS AVAILABLE FROM MODEL -> OPEN.....	27
3.2	EXPORTING AND IMPORTING CORMAS MODELS.....	27
4	CORMAS VISUALISATION'S TOOLS.....	28
4.1	SPATIAL INTERFACE.....	28
4.1.1	The Tessellation submenu.....	29
4.1.2	Topology.....	29
4.1.3	Tools.....	30
4.1.3.1	Click to change attribute... versus Click to inspect.....	30
4.1.3.2	Saving and loading the whole spatial grid.....	31
4.2	COMMUNICATION INTERFACE.....	31
4.3	CHARTS INTERFACE.....	32
5	RUNNING SIMULATIONS WITH CORMAS.....	34

1 GETTING STARTED

Cormas is a simulation tool based on the programming environment VisualWorks, a software used to develop Smalltalk applications. The installation procedure involves two steps : download the non-commercial version of VisualWorks 3.0, then download Cormas.

1.1 INSTALLING VISUALWORKS

CinCom recently bought VisualWorks from ObjectShare and released a new version of VisualWorks (5i.3). A non-commercial version of VisualWorks (for evaluation and educational purposes) is available¹. By now, Cormas is compatible with a previous version of VisualWorks (3.0) which must be installed before Cormas can be used (a Cormas version compatible with this latest VisualWorks release is expected by the end of 2001).

1.1.1 DOWNLOADING VISUALWORKS 3.0 NON-COMMERCIAL

The VisualWorks non-commercial 3.0 release is distributed via the web. You can retrieve it directly from: **<http://wiki.cs.uiuc.edu/VisualWorks/DOWNLOAD>**

1.1.2 UNZIPPING AND MOVING VISUALWORKS 3.0 NON-COMMERCIAL FILES

Installing VisualWorks is a simple matter of unzipping the downloaded files.

Extract all the files from the WinZip archives ("Use folder names" should be checked to ensure the creation of the subdirectories structure during the extraction). The files should be organised in a directory structure that is assumed by the initial configuration of VisualWorks. Some files, such as the object engine and the image, are expected to be in specific directories (respectively *\bin* and *\image*) relative to the root installation directory *\vwnc30*. If you need to move the *\vwnc30* installation directory, including all of its files and subdirectories, to a directory on your installation disk, please be aware that the target directory must not have spaces, or be created as a subdirectory in a path that has spaces (e.g., *Program Files*), in the name.

¹ <http://www.cincom.com/smalltalk/vwdownload.asp>

1.2 INSTALLING CORMAS

The Cormas distribution is compound of a set of seven .st files which altogether represent a VisualWorks application, and a library of models built with this application.

1.2.1 DOWNLOADING CORMAS

Cormas is available on the web at <http://cormas.cirad.fr/>

1.2.2 UNZIPPING AND MOVING THE CORMAS FILES

Create a `\cormas` subdirectory relative to the VisualWorks 3.0 Non-Commercial root installation directory (`\vwnc30`).

Move the downloaded archive *Cormas.zip* into this directory and extract all files ("Use folder names" should be checked in WinZip to ensure the creation of the subdirectories structure during the extraction). The following 3 subdirectories should appear:

- Kernel*
- Messages*
- Models*

1.2.3 COPYING THE BASIC VISUALWORKS IMAGE

Copy the *visualnc.im* file from `\vwnc30\image` to the Cormas installation directory (`\vwnc30\cormas`)

1.2.4 OPENING VISUALWORKS

Double-click on the *visualnc.im* file in the cormas subdirectory. If the operating system doesn't know which application to use to open this kind of files (Smalltalk images), create a new association by clicking on **other** and specifying `\vwnc30\bin\visualnc.exe`

1.2.5 ADJUST THE VISUALWORKS HOME DIRECTORY

In the main menu of the VisualWorks Launcher, select
File --> Set VisualWorks Home

Enter the path to the root VisualWorks installation directory. A directory is proposed based on the startup directory. Verify that it is correct (`\vwnc30`), and change it if necessary. Then click **"OK"**.

This sets the value of the *VISUALWORKS* variable used by VisualWorks to access source files using the portable filename mechanism.

1.2.6 ADDING REQUIRED PARCELS

In the main menu of the VisualWorks Launcher, select

Tools --> Load Parcel Named...

A prompter appears requesting the name of the parcel file. An asterisk ("*") should be showing in the input field. Leaving the "*" as is, press the **OK** button. The directory registered in *\vwnc30\parcels* path will be searched, and all readable parcels will be listed. Select the items

BGOK 3.0 ncuo
ColorEditing 3.0 ncuo
JuggleButtons 3.0 ncuo
Lens3.0.0 3.0 ncuo
RefactoringBrowser 3.0 ncuo
UIPainter 3.0 ncuo

and press the **OK** button, then **Yes to All**.

1.2.7 ADDING CORMAS SPECIFIC TOOLS TO THE VISUALWORKS LAUNCHER

In the Workspace window, type the instruction:

```
'c:\vwnc30\cormas\kernel\VisualLauncher.st' asFilename fileIn.
```

Highlight the text with the mouse, and then execute it by choosing the “**do it**” item from the contextual menu appearing after a right click (on the right button of the mouse) or a <CTRL-right click> (cf. next operation in 1.2.8).

In the main menu of the VisualWorks Launcher, select

Tools --> Cormas --> Install Cormas

1.2.8 ADJUST THE VISUALWORKS SETTINGS

In the main menu of the VisualWorks Launcher, select

File --> Settings

- Select “**UI Options**” and check “**Globalization**”, then “**Accept**”.
- Select “**Source Files**”. Select the second filename. Correct to:
\$(VISUALWORKS)\cormas\visualnc.cha
then “**Change**” and “**Accept**”.
- Select “**Messages**”. Enter:
\$(VISUALWORKS)\cormas\Messages
then “**Add**” and “**Accept**”.

- If you get the contextual menu with a <CTRL-right click> (see 1.2.6), and prefer to get it directly with a right click, select “**Mouse buttons**”, toggle the check box, and “**Accept**”.

1.2.9 SAVE THE VISUALWORKS ENVIRONMENT SET UP FOR CORMAS

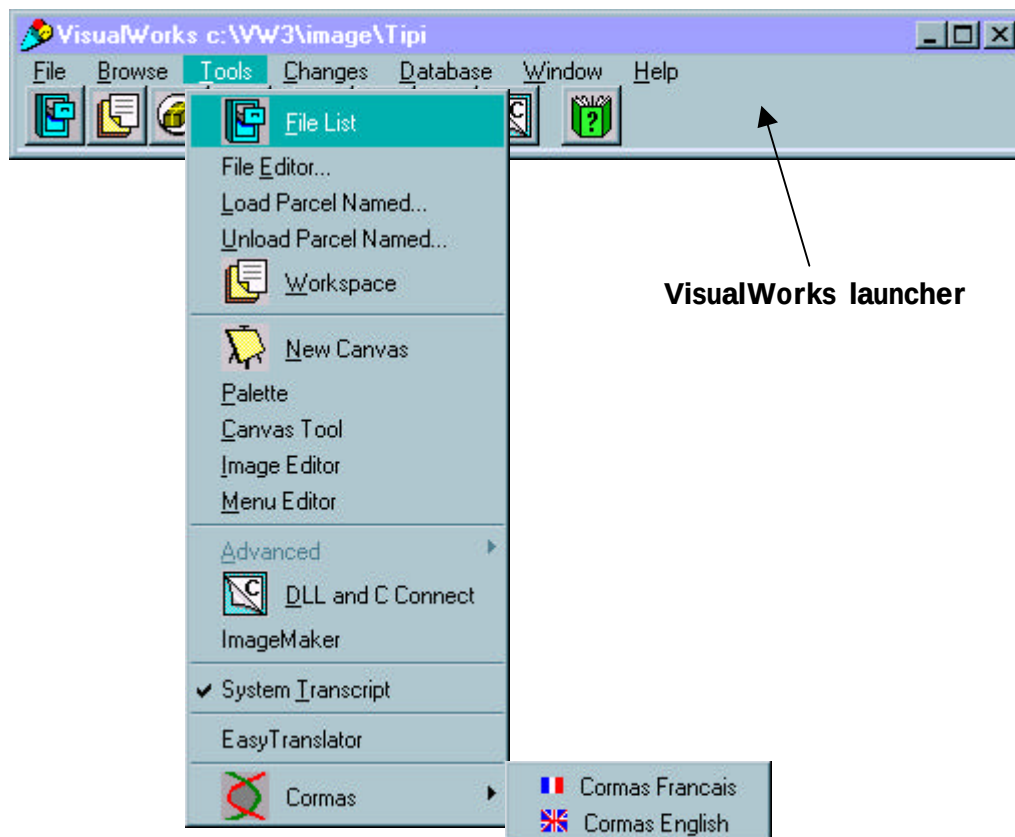
In the main menu of the VisualWorks Launcher, select
File --> Save as

Change the suggested name to:
c:\wwnc30\cormas\cormas2001

1.3 OPENING CORMAS

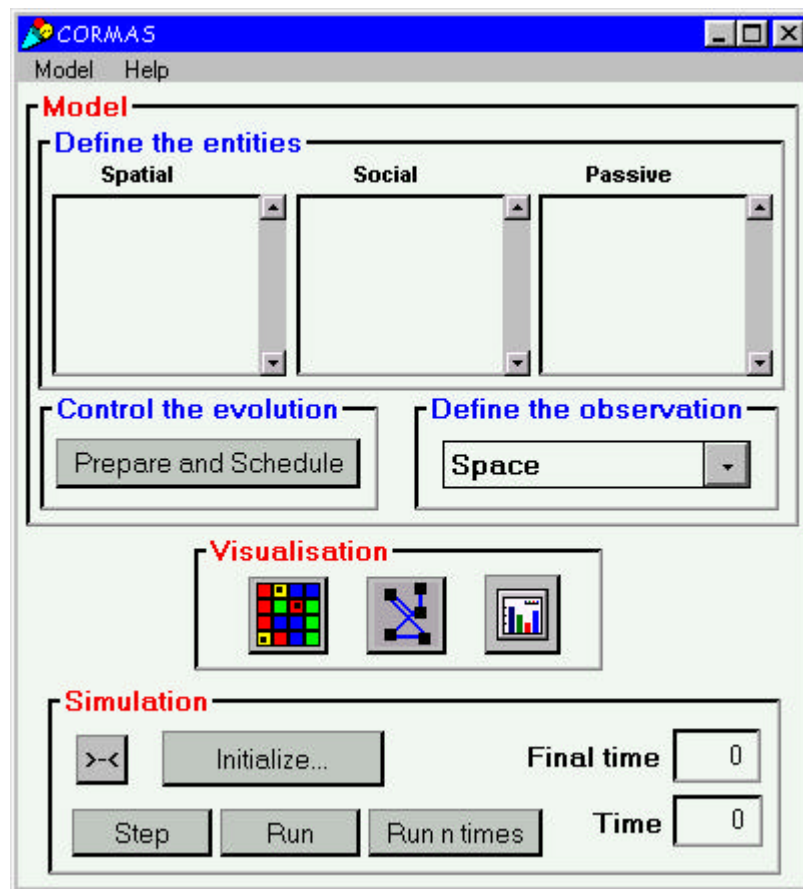
In the main menu of the VisualWorks Launcher, select
Tools --> Cormas --> Cormas English

Or... Dans le menu de la fenêtre principale de VisualWorks, sélectionner
Tools --> Cormas --> Cormas Français



Cormas is a tool made to build models and to run them. The Cormas main interface is divided into three parts. The upper one deals with the definition of the model, the

lower one deals with the simulation of the model, and between them, they are three visualisation's tools.



These three parts are described in the next chapters. Chapter 2 gives the elements to build a model with CORMAS. Chapter 3 presents the three visualisation tools: the first one deals with the spatial representation of the model, the second one deals with the observation of the communication, the last one describes how to easily plot simple x-y charts.

2 BUILDING A MODEL WITH CORMAS

Within Cormas, the modelling process is divided into three parts :

- The definition of the model's entities, located in the upper part of the model box, is based on a classification into 3 categories: spatial, social and passive.
- The control of the global evolution of the model. In the lower left part of the model box, a button labelled “**Prepare and Schedule**” opens a window used to define the model's class itself.
- The definition of the model's observation. In the lower right part of the model box, a scrolling menu button (with “**space**”, “**communication**” and “**charts**” items) allows to open some specific tools to define viewpoints on the model.

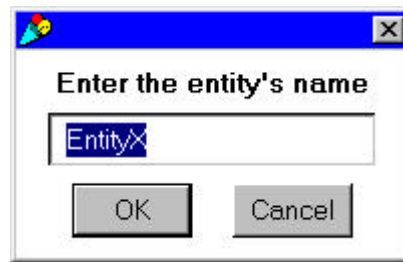
2.1 DEFINITION OF THE ENTITIES

2.1.1 HOW TO CREATE, MODIFY OR DELETE ENTITIES

The edit menu for the 3 entities' categories is a contextual menu of the 3 lists (accessible via a right-click).

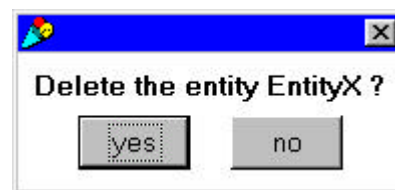


When the “**Add**” item is selected, submenus enable to choose the appropriate superclass from which the new entity will inherit. This choice, that depends on the kind of entity (spatial, social or passive), will be described in details below, in specific sections. Once this choice has been made, a window appears, requesting a name :

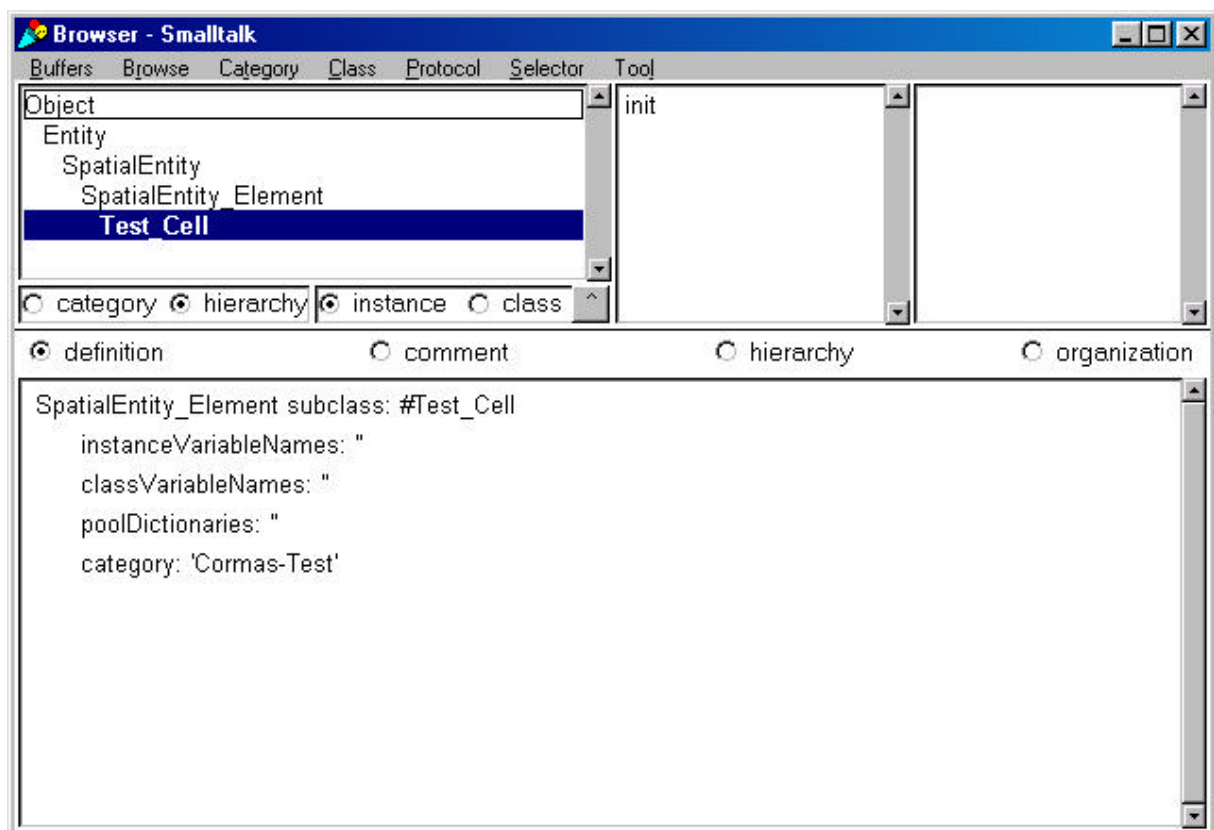


The name entered should not already exist as a class name in the whole VisualWorks programming environment.

The “**Delete**” and “**Modify**” items are disabled if there is no selected entity in the list. A confirm request dialog is associated to the delete action.



Once a new entity is added or modified, the entity definition window appears. It is based on the RefactoringBrowser tool.



In this RefactoringBrowser, the “hierarchy” view being selected by default, one can retrieve the name of the superclass in the editor and in the list of inheriting classes.

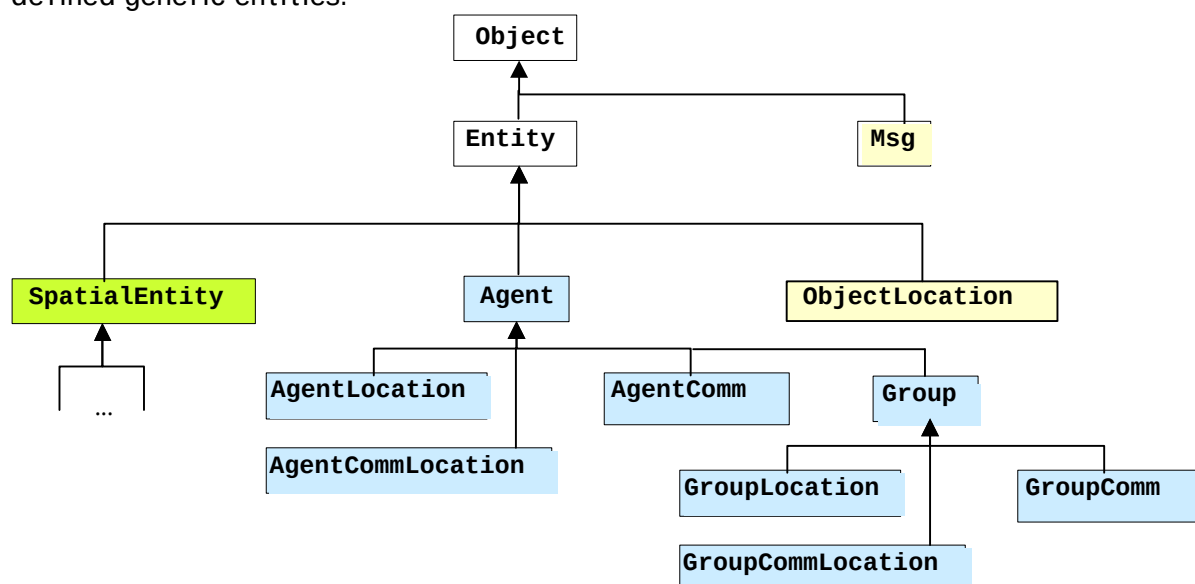
For spatial entities, a protocol **“init”** is automatically created, with a single available method also named **“init”**. This method simply calls the superclass **“init”** method. It has to be completed if needed.

For social entities, in addition to this **“init”** protocol defined in the same way, a protocol **“id”** and a protocol **“control”** are also automatically created. The **“step”** method that belongs to the **“control”** protocol has to be completed. It is a generic name for a method that will be used at each time-step during a simulation. The **“id”** protocol is used to let the class automatically delivers unique and incrementing values as id numbers to be assigned to the **“id”** attribute of newly created instances. This mechanism is related to the generic method **“initAgents”** of the **“Cormas_Model”** class which empties the collection of entities and resets the current **“id”** value to zero.

For passive entities, nothing is created by default.

2.1.2 THE CORMAS CLASSES' HIERARCHY AND THE ROOT CLASS *Entity*

Taking advantages of the object-oriented programming, Cormas proposes a set of pre-defined generic entities.



The three categories of spatial (left, for their detailed hierarchical organisation see section 2.1.3), passive (right, section 2.1.4) and social (middle, see section 2.1.5) entities are represented with their inherited relationships. The root class of this hierarchy is the *Object* class, which is the most general and abstract class of all the SmallTalk classes.

Cormas is a multi-agent platform proposed to implement simulation models for resource management. Some programmes have been prepared for that purpose. These tools are proposed to simulate a relation between the request and the offer of resources. This

may be implemented with the messages and communicating agents but we did not wanted to use the social communication metaphor. Thus we have proposed a simple implementation more based on a metaphor of physical harvest, which has been implemented at the highest level of the Cormas hierarchy (within the *Entity* class). Two main rules have been implemented for sharing the resources, either to account for an asynchronous mode ("first asking, first served") or to account for a synchronous mode (each agent receives an amount proportional to its request). Any entity of Cormas has two attributes *request* and *qtyGiven*. When an entity wants to harvest a resource held by another entity, it uses the following message:

```
<receiver_entity>
  receiveRequestAbout: <attribute>
  qty: <q>
  from: <sender_entity>
```

The sender (usually *self*) asks for a quantity of resource held by the receiver in an attribute. To use this programme the sender and the receiver must have the same attribute. The receiver will stock this kind of request in its attribute *request*. It is a collection of triplets (attribute, quantity, sender). Then, once all the requests have been done, the entity will share the resource. Different programs are available.

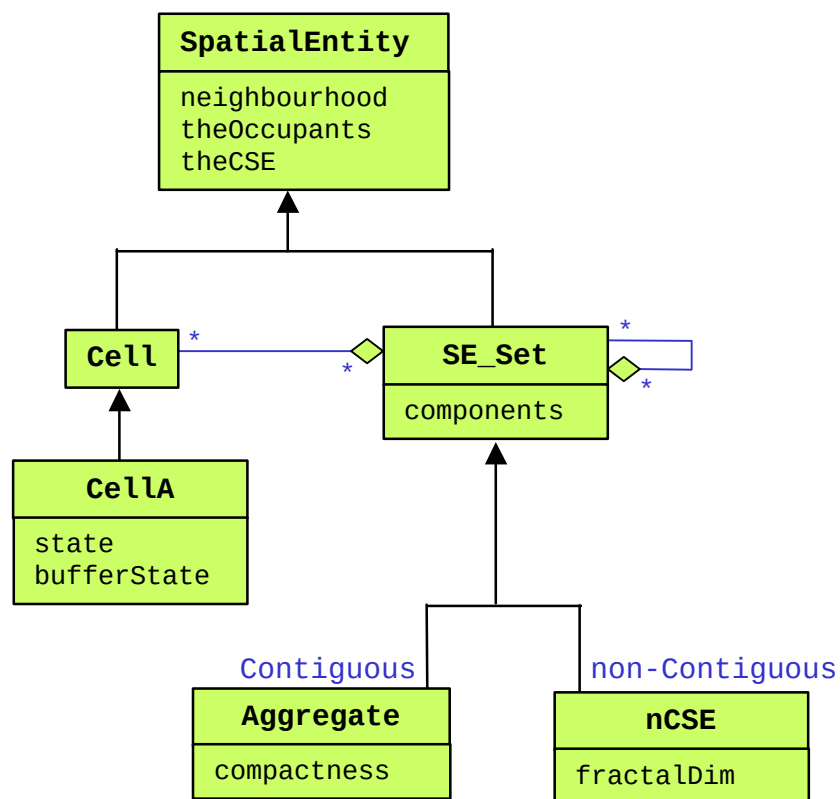
- *treatRequestOrder* gives the resource according to the order of reception. First request, first served.
- *treatRequestProrata* shares the resource between the senders depending on the quantity requested.

For a better understanding of these mechanisms, choose within the Cormas' library the model labelled **TestResourceExchange**.

2.1.3 SPATIAL ENTITIES

The construction of a spatial support with Cormas goes through the definition of an elementary spatial entity, which will represent the smallest homogeneous portion of the environment in the model. The identification of the appropriate spatial resolution is one of the key-feature of the modelling process. It is directly correlated to the objective of the model.

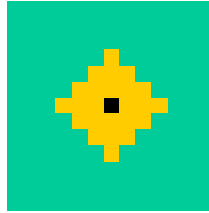
The hierarchy of spatial entities proposed by Cormas is based on a general abstract root class, named *SpatialEntity*, from which the elementary spatial entity (*SpatialEntity_Element* or *Cell*) directly inherits.



At the abstract level of *SpatialEntity*, some attributes and methods shared by every kind of Cormas spatial entities have been defined. For an exhaustive and detailed description, see the appendix to this user's guide).

- Some attributes are related to the topology (*center*, *outline*, *neighbourhood*, *extensiveNeighbourhood*). The *neighbourhood* attribute is defined as a collection of adjacent instance of the same class. Its size will depend on the topology of the spatial grid (see section 3.1.2). The *extensiveNeighbourhood* attribute store a larger collection. The usual way to define an extensive neighbourhood is to recursively look for neighbours of the adjacent neighbours.

There is a method (*recursiveNeighbourhood: n*) which computes recursively this extensive neighbourhood (within a area of radius *n*) and returns it. Look at the example below with *n=3* in the case of topology based on squared cells with 4 adjacent neighbours.



Some Cormas models use this extensive neighbourhood to define agents' wider perception range. See for example the *Djemiong_Duiker* class of the **Djemiong** model.

- Some attributes are related to the displays offered by the visualisation's tools (*image, view, visualState*).
- *theOccupants* is a register of all the inhabiting situated agents or passive objects,
- *theCSE* registers the belonging to composed spatial entities

The hierarchical organisation of the Cormas generic spatial entities is based on this last feature. Basically, each class of Cormas spatial entity is susceptible to be defined as a spatial component any other Cormas compound spatial entity. All the classes inheriting from *SpatialEntitySet* have a *components* attribute. When a compound spatial entity is defined or evolves, its components attribute is updated by using one of the methods *addComponent:*, *addComponents:*, *deleteComponent:* or *deleteComponents:*. Within these methods, each element of the components that has been added or deleted updates its attribute *theCSE*. A method implementing exchange of components between compound spatial entities is based on these primitives:

```
receiveComponents: aCollec from: anotherCompoundSE
    anotherCompoundSE deleteComponents: aCollec.
    self addComponents: aCollec
```

By using these generic method, the updating of the both-ways referencing mechanism based on the couple of attributes *components/theCSE* (similar to the one linking the *cell* and *theOccupants* attributes, as mentioned below) is transparent to the user.

The spatial entities inheriting from the abstract class *Aggregate* are built by aggregation of lower level contiguous spatial entities. The spatial environment of CORMAS provides generic methods that perform this kind of aggregation. Thus for the construction of *Aggregate* spatial entities, the following three methods are available:

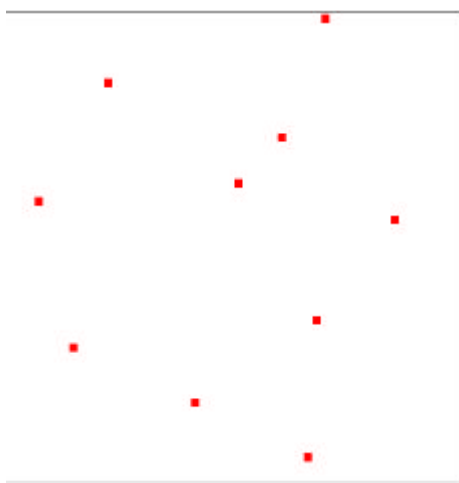
```
partitions: <entityC> madeOf: <entityE> attribute: <attributeName>
    where <entityC> and <entityE> are respectively the higher and the lower levels
    spatial entities, and <attributeName> is the name of the attribute of the lower
    level spatial entity on which the aggregation is processed. The result will give as
```

many distinct instances of Partitions as there exists distinct values for this attribute in the whole collection of lower level spatial entity.

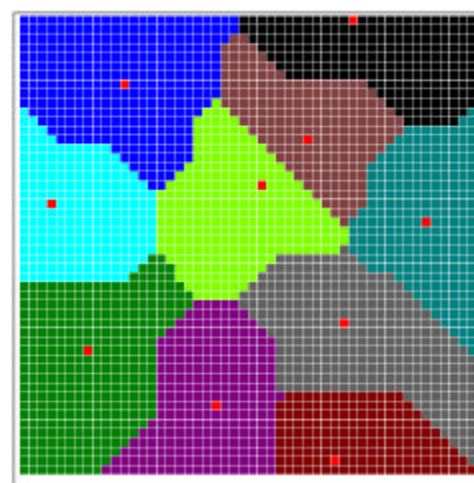
partitions: <entityC> fromSeeds: <aCollec>

where <entityC> is the higher level spatial entity, and <aCollec> is a collection of instances of the lower level spatial entity. Starting from these kind of seeds, the aggregation is processed by recursively looking for not yet aggregated instances of the lower level spatial entity in the outside surround of the newly created instances of the *Aggregate* class.

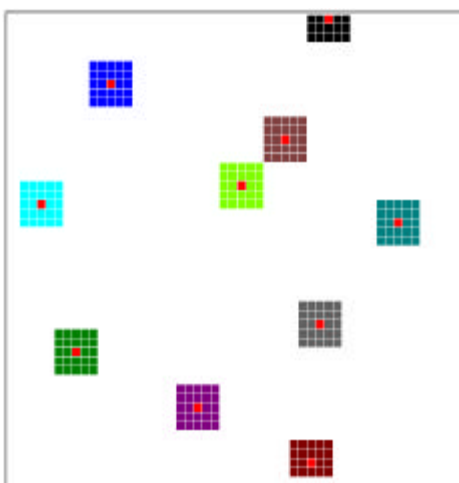
(a) (b) and (c) figures below illustrate this kind of progressive extension by diffusion from an initial random distribution of 10 seeds in a 30*30 spatial grid (a), after 2 recursions of the algorithm (b), and when the process is completed (c).



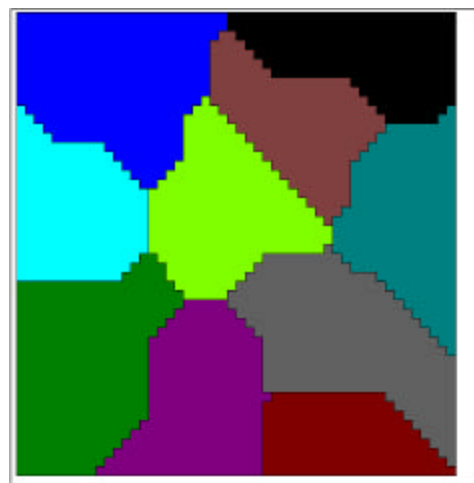
(a)



(c)



(b)



(d)

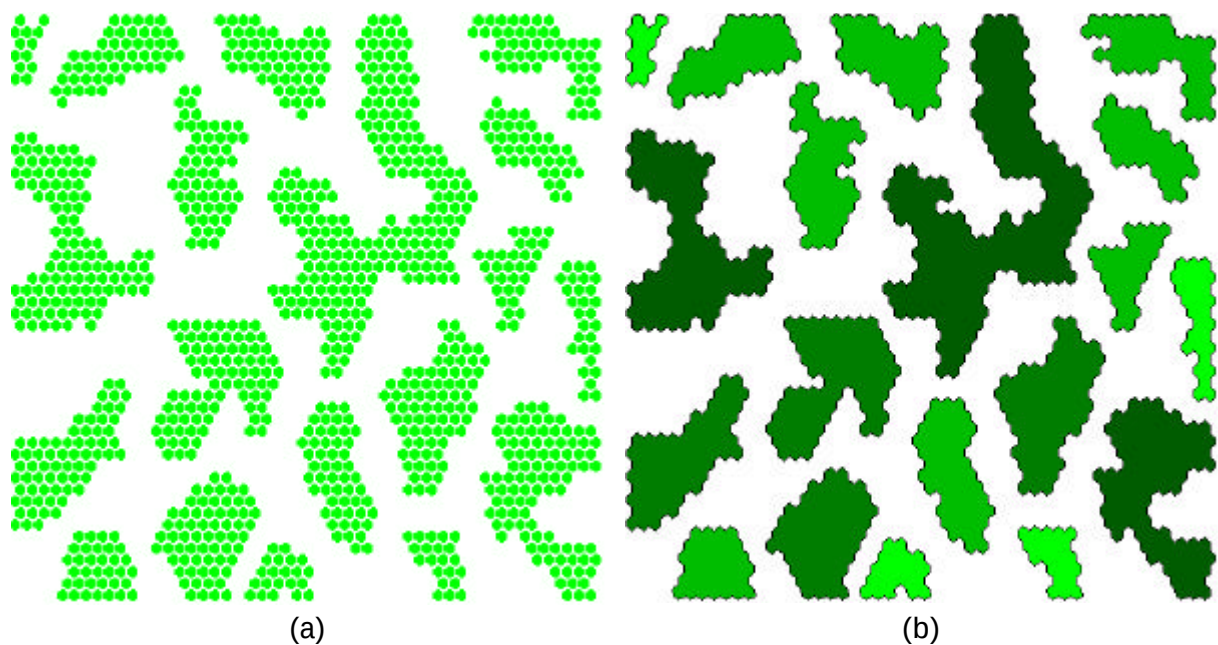
Another point has to be noted from figure (d): the definition of points of view is available for all the CORMAS spatial entities of a model. The polygonal image of a compound spatial entity is automatically adjusted from the outlines of the lower level spatial entities belonging to its inside surround, and the colour used to display the compound entities are based on the visual state specified in the active point of view

method, in the same way than all the other CORMAS entities. In the example showed in figure (d), the colour has been randomly assigned when the *Aggregate* instances were created.

Methods corresponding to situations where the aggregation is conditioned by the state of the lower level spatial entities, this state eventually fluctuating dynamically during the simulation, are also available. The generic method provided by the spatial environment of CORMAS stands as follow:

aggregates: <entityC> madeOf: <entityE> condition: <methodName>
where <entityC> and <entityE> are respectively the higher and the lower levels spatial entities, <methodName> is the name of the condition for any instance of the lower level spatial entity to be aggregated.

Below this method is illustrated in a 30*30 spatial grid made of hexagonal cells with a very simple state attribute being either #empty (white) or #forest (gray) (a).



The result of the aggregation of the # forest cells is materialised in figure (b) with the viewpoint on the compound spatial entities based on their size (the darker, the bigger).

For a better understanding of the use and definition of these composed spatial entities, have a look at the CORMAS model “TSE”.

2.1.4 PASSIVE ENTITIES

There are two categories of passive entities. The first one concerns passive objects, the second one is about messages.

2.1.4.1 The passive objects of Cormas

When the passive object is of any kind that do not need to be situated on the spatial grid, then the superclass is directly the Smalltalk root class "*Object*". For situated passive object, there is a particular Cormas class named *ObjectLocation*.

For situated passive objects, a specific method, (*isMovedTo:*) has been defined, to account for change of location. Contrary to the situated agents, instances of *ObjectLocation* are not able to find by themselves their destination, neither to perceive their local environment (see section 2.1.5).

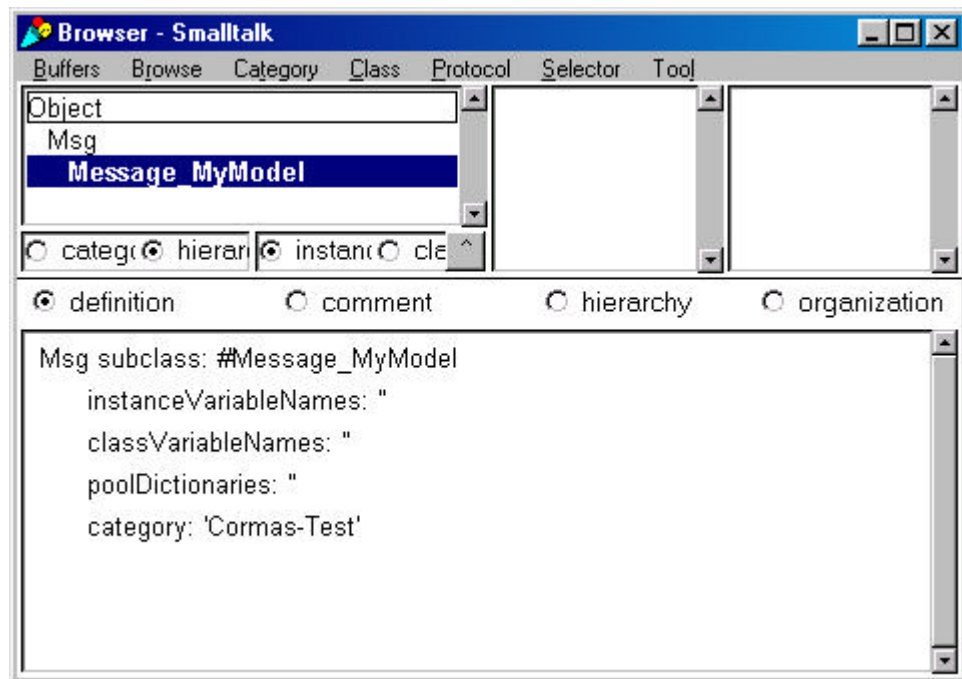
2.1.4.2 How to create a message ?

On the list "**Passive entities**", the contextual menu "**Add Message**" is disabled until a communicating agent has been created.

Type the name of the message class :



The message definition window appears :



Have a look at the superclass "**Msg**", one of the Cormas generic entity, to see the three basic attributes of a message :

- *sender* : is used to set the sender of the message,
- *receiver* : is used to set the receiver of the message,
- *symbol* : is used to define the type of message.

The modeller can add any kind of attributes to be used in more complex conversation. For instance in the example shown figure 8, two attributes (*object* and *amount*) have been added for a merchant exchange protocol (see the "**PlotsRental**" model in the Cormas library).

2.1.5 AGENTS

Multi agent systems are useful for resource management for several reasons. Firstly MAS are used to represent agents moving and harvesting resources on a spatial environment. Cormas proposes a class ***AgentLocation*** that implements programs to move on a spatial context. Secondly, MAS are used to simulate co-ordination between agents through communications. The class ***AgentComm*** has been created for that purpose. The class ***AgentCommLocation*** offers altogether the methods for spatial movements and communication. Lastly MAS are used to study the links between various organisation levels : the *Group* class and its subclasses (communicating group, situated group, communicating and situated group) have been created.

2.1.5.1 AgentLocation

Programs have been implemented in this class to give the agents spatial references. The agent is situated on a spatial entity. Its *patch* attribute contains the spatial entity instance. Two main methods have been programmed to model movements.

1. The *leave* method breaks the link between an agent and its spatial reference. The attribute *patch* of the agent is set to nil, and the spatial entity removes the agent from its attribute *theOccupants*.
2. The *moveTo: aDestination* links the agent and the object *aDestination* which is a spatial entity. The *patch* attribute of the agent is set to *aDestination*, which adds the agent in its attribute *theOccupants*.

```
<located_entity> leave. <located_entity> moveTo: aSE
```

Usually the main method of such an agent ends with these two instructions. The previous code aims at defining which spatial entity will be chosen. The *randomWalk* method gives a good example of a simple implementation

```
randomWalk  
| destination |  
destination := Cormas selectRandomlyFrom: self patch neighbourhood.  
self leave.  
self moveTo: destination
```

2.1.5.2 AgentComm

An instance of *AgentComm* class or an inherited class possesses two main attributes *channel* and *mailBox*. These attributes are implemented to allow communication. A communicating agent is linked to a communication channel (the attribute *channel*). This channel is in charge of managing communications. The modeller does not have access to

the channel. To implement communications the modeller just has to make agents read their *mailBoxes* and send *messages*.

- Basic steps to send a message

1. Create an instance of the required message class

In this example, its name is *MessageClass*, which has to be a subclass of the generic entity *Msg* (see section 2.1.4.2):

```
<instance_message> := <MessageClassName> new.
```

2. Write the instruction :

```
<instance_message> symbol: #<information_object>;  
sender: <sender_entity>;  
receiver: <receiver_entity>.
```

Where :

- #<information_objjet> is a symbol for the type of message;
- <sender_entity> the emitter of the message (usually *self*);
- <receiver_entity> the entity to whom the message is addressed.

Additional attributes defined by the modeller have also to be fulfilled.

3. Send the message:

Two methods are available depending on the scheduling of the model. In asynchronous mode the message is immediately delivered to the receiver's mailbox. In synchronous mode, the message is delivered at the end of the time step, once all entities have sent their messages.

```
<sender_entity> sendMessageAsynchronously : <instance_message>.  
<senedr_entity> sendMessageSynchronously : <instance_message>.
```

- How to process a message ?

To process messages Cormas uses a *mailBox* which contains all the received messages. *mailBox* is an attribute of a communicating entity. It is initialised as a collection, therefore the modeller will use the associated methods such as :

- *do*: to process successively all the messages
- *select*: to select messages depending on a criterion to be specified
- *isEmpty* to test whether the mailbox is empty or not

A method *evolve* of the communicating entities is proposed to read the messages. It collects the messages sequentially and remove them from the mailbox. The modeller has to define its own method *processMessage*.

2.1.5.3 Group

MAS are useful for simulation of ecosystems or societies modelling because it is possible to simulate interactions between entities across different scales and organisation levels. A *Group* class has been implemented to allow the representation of composite social entities. The main attribute of the *Group* class and inherited classes is the attribute *components*. It is initialised as a set. The modeller may add and remove components by simply using the following methods :

```
<group_entity> addComponent: anElement  
<group_entity> addComponents: aSubset  
<group_entity> deleteComponent: anElement  
<group_entity> deleteComponents: aSubset
```

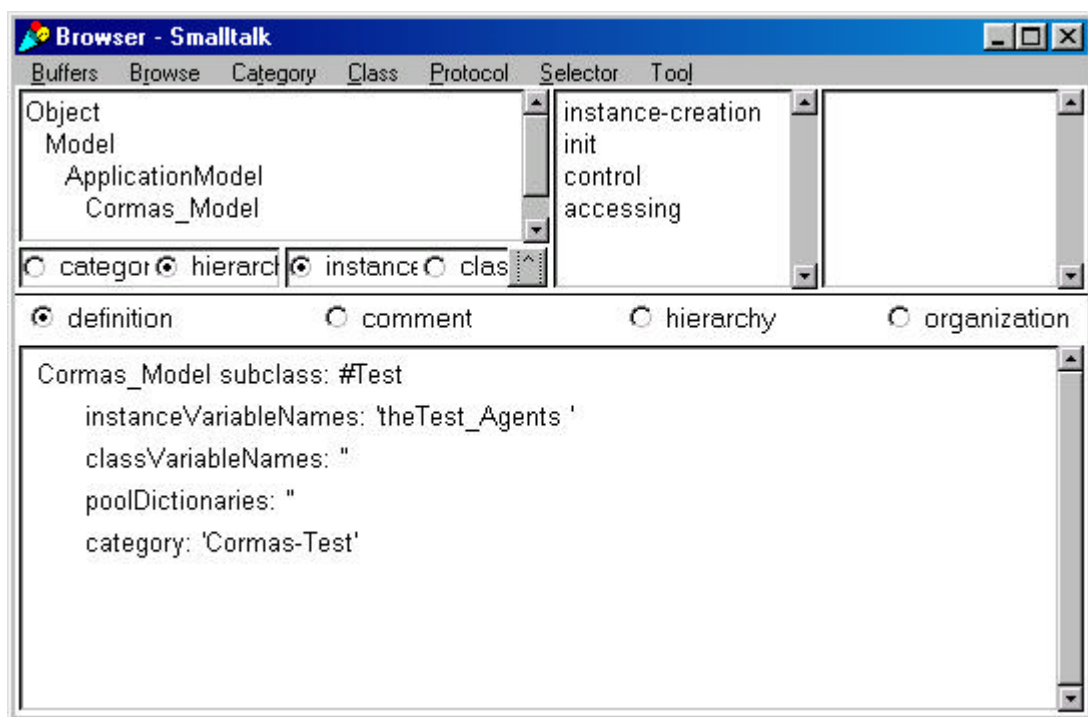
The components may be added and removed in a synchronous or asynchronous mode. The previous methods are used for asynchronous scheduling. For synchronous simulations the group composition will be updated at the end of each time step. For that purpose two attributes have been added, *newComponents* and *removedComponents*. Instead of adding or removing the components directly in the *components* attribute, these attributes are used as buffers.

```
<group_entity> synchronousAddition: anElement  
<group_entity> synchronousRemoval: aSubset
```

At the end of each time step the modeller has to activate the *updateComponents* method.

2.2 CONTROLLING THE EVOLUTION OF THE MODEL

Once the modeller has defined all entities of its model, he can move on to the initialisation of the model and its control. Cormas proposes a special class : *Cormas_Model*. When a new model is defined, a class inherited from this generic class is automatically created. The name of this class is the name of the model. For that reason we often call this class the model controller. On the Cormas interface, lower-left part of the model box, click on the button "**Prepare and schedule**". A window which is quite similar to the entity's definition window opens.



As the model is supposed to work with populations of entities that have been defined just before, some attributes have been automatically created. For an entity named **Test_Agent** an attribute of the model will be *theTest_Agents*. It corresponds to a collection of instances of this entity. Thus the model's controller will be able to schedule the activities of all the entities.

Additionally, four protocols have been automatically created.

- "**accessing**" groups together the accessors methods.
- "**init**" has a single method also named "**init**" that proposes a general framework to initialise the model. This method is using three basic methods defined at the level of the superclass *Cormas_Model*. Firstly the initialisation of the cells (**initCells**), then initialisation of the agents (**initAgents**), and finally the initialisation of the charts (**initCharts**).

- “**instance-creation**” has a single method (**initAgents**) that has to be completed from the generic one: the “*super initAgents*” instruction empties all the collections of agents (in our example *theTest_Agents*) and also resets the current id value for each class of agent to zero. The effective creation and initialisation of agents have then to be written after this first instruction.
- “**control**” has a single method named “*step:*”, that has to be completed. This method will schedule the interactions between the entities. It always has to take the time-step of the simulation as an argument, which is needed to update the charts with the generic “*updateCharts:*” method.

2.2.1 INITIALISATION OF A MODEL

The modeller has to write a program to create new instances of entities and to make links between these entities. For instance a situated agent's instance will be linked to a spatial entity's instance. A standard initialisation method is proposed at the level of the *Cormas_Model* class. The modeller can use it as a framework to write the method specific to its model.

1. The first step of an initialisation method consists in defining the basic spatial entities. The method *initCells* is the standard one to get all the elementary spatial entities created with the spatial interface (see section 3.1) and to initialize them (by default with their *init* method, if another one is needed, it has to be given as the attribute of the *initCells:* method).

```
self initCells. or: self initCells: #initSpecial
```

2. Then for each entity *EntityX* of the model, the attribute *theEntityXs* has to be set up as a collection. Again there is a generic method, *initAgents*, which is processing the basic operations. A classical way to add to this method the specific instructions is to redefine it that way:

```
initAgents
  super initAgents.
  n timesRepeat: [
    a := EntityX new init.
    self theEntityXs add: a]
```

3. To initialize the charts, use the instructions *self initCharts*

2.2.2 CONTROL OF THE MODEL

The modeller has to define a method which implements the organisation of a time step. The purpose is often to loop on each collection of entities to activate the entities' main method of evolution.

A control procedure must have an argument which is the time step. For instance the method will be named *step: t*.

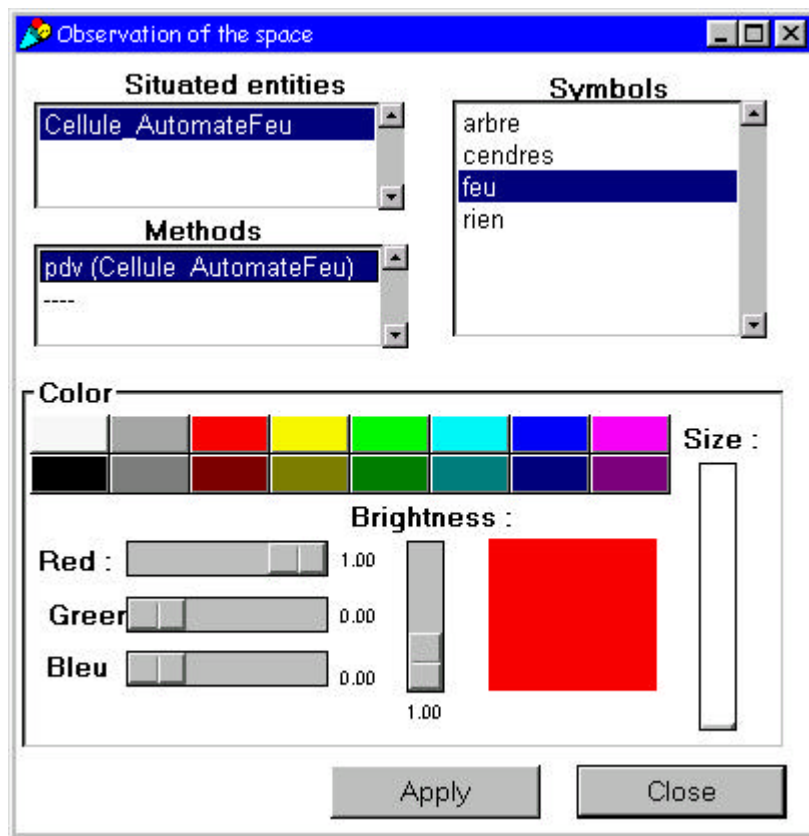
The simulation interface proposes to run multiple simulations “**Run n times**”. In case of multiple simulations the method will need two arguments, the first one for the time step and the second one for the number of simulations to be carried out. For instance the method could be named *step: t nTimes: n*

2.3 HOW TO OBSERVE THE MODEL

Once entities of the model have been created and a controller class has been implemented, the modeller needs to define its own points of views for the observation of the global system. MAS are tools used for a bottom-up approach. The global properties of the system emerge from the interactions of the entities. These properties depend on the point of view of the observer. Cormas offers three kind of interface for the observation: observation of the spatial dynamics, of the communication dynamics and the classical scientific graphs. These interfaces are accessible via the scrolling menu button located in the lower-right part of the model zone, within the Cormas main interface.

2.3.1 OBSERVATION OF THE SPACE

An interface is proposed for spatial dynamics. This interface represents spatial entities and situated objects and agents. Depending on the state of these entities, a point of view may be defined. A state is the value of an attribute or the combination of several values. This point of view is a Smalltalk method to be implemented by the observer. For each state of an entity, the point of view method associates a symbol. This is done when an entity runs the method *defineVisualState*. For spatial entities, this symbol refers to a color. For situated entities, this symbol refers to a shape, a color and a size. An interface is proposed. On the Cormas interface select “**Space**” in the “**Define the observation**” scrolling menu button. A window opens.



2.3.1.1 Definition of a point of view.

A point a view is a method to be written by the modeller. First, select the entity, then from the contextual menu of the list **"Methods"**, choose **"Add"**. A code-editor window is opening. The principle is to associate a state of the entity to a symbol. The output of the method must be a symbol. For instance, a simple point of view method looks like:

```
povElections
self state = 0 ifTrue: [^#democrats].
self state = 1 ifTrue: [^#republicans].
```

2.3.1.2 Definition of a symbol

A symbol represents the state of a spatial entity, i.e. the value of an attribute or a combination of values. To define symbols, from the contextual menu of the list **"Symbols"**, choose **"Add"**, then enter the name of the symbol (do not type the # character) and click **"Ok"**.

2.3.1.3 Definition of the shape, size and color.

Each symbol is associated to a shape, a size and a color. The basic steps to establish this association stand as follow:

1. Select the entity
2. Select the point of view

3. Select the symbol.
4. Use the sliders (or the palette) to define the color and the brightness.
5. For agents you can also define the shape of the symbol. On the window representing the shape, open the contextual menu. You may choose empty or filled polygons, define the number of segments. For empty polygons you may choose the size of the line. Then select the size of the shape with the bottom-right slider.

2.3.2 OBSERVATION OF COMMUNICATION

A second interface is proposed to observe communications between the agents. The modeller can choose what kind of communications will be displayed on the interface. On the Cormas interface select “**Communication**” in the “**Define the observation**” scrolling menu button. A window opens.

When “**Erase links at each time step**” is checked, the communication graph will be erased at the end of each time step. In the list “**Methods**”, the modeller may add methods and have to select one of them to specify what kind of messages will be represented on the interface during a simulation. Communication often generate many messages, and it can be useful to select some of them. The modeller will write a program based on the state of the message. The output of the method must be a boolean value (true or false). Two methods are proposed :

always
 $\wedge true$

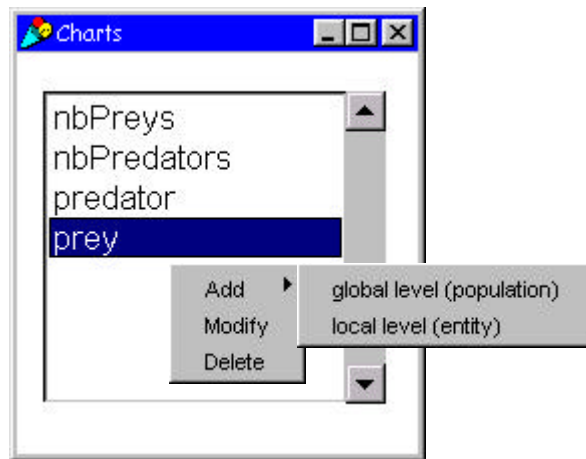
never
 $\wedge false$

The methods the modeller adds are often based on a test on the symbol of the message but may also be related to any message attributes.

2.3.3 CHARTS

The third interface proposed concerns charts. After running the simulation the modeller often wants to look at the evolution of indicators. Cormas provides facilities to design line-charts with x-values being the time-steps of the simulation, and y-values any value (which is returned by a method that the modeller has to write).

On the Cormas interface, select “**Charts**” in the “**Define the observation**” scrolling menu button. A window with the list of charts opens. With the contextual menu, addition, modification or removal of charts are enable.



Two levels of data-recording are available. The first one is the global level, the y-values are then typically the size of agents populations. The second one is the local (individual) level, allowing to track all the instances of a given agent class.

When adding a global charts, a block-editor is opening to write a method called *chartNameData* that should return the value to be recorded. For instance in the **Pursuit** model, the global chart *nbPreys* has the corresponding *nbPreysData* method:

```
nbPreysData
"return the data (a number) to be plotted with the nbPreys chart"
^self thePursuit_Preys size
```

When adding a local chart, the procedure is similar, but the data to be returned should be a value computed as many times as there are instances of the entity which is tracked. The *chartNameData* method should here return an array made of two symbols. The first one has to be the entity class name and the second one the name of the method (to be defined at the entity level) that should return the data (a number) to be plotted with the chart. In the **Pursuit** model, the global chart *preys* has the corresponding *preysData* method:

```
preysData
^#(#Pursuit_Prey #isDead)
```

3 OPENING, IMPORTING, EXPORTING MODELS

3.1 MODELS AVAILABLE FROM MODEL -> OPEN

Once you have built your own model, when you save the VisualWorks image (which should be named *cormas.im*, cf. session 1.2.9), your model is saved within this file. Actually when a new model (named for instance **MyFirstCormasModel**) is created with Cormas, a new category named **Cormas-MyFirstCormasModel** is automatically created in VisualWorks. You can find the model's classes created with Cormas in a VisualWorks SystemBrowser: they are all grouped together within this category.

The next time you will open the *cormas.im* file and run Cormas, your model will belong to the list of models proposed by the **Model -> Open** option from the cormas main menu. To build this list, Cormas is looking for all the VisualWorks categories with a name matching **<Cormas-*>**.

3.2 EXPORTING AND IMPORTING CORMAS MODELS

Another possibility to save your model is to use the **Model -> Export** option from the cormas main menu. By doing that, you create or update two ascii-files in a subdirectory *\Models\MyFirstCormasModel*, the first one (*MyFirstCormasModel.st*) with the source code of your model, the other one (*MyFirstCormasModel.ev*) with the visual states created for the observation of your model. This way is useful for exchanging models. The maps eventually used to initialize (see section 4.1.3.2) or to save (see section 4.1.4) the spatial grid are also saved within this subdirectory (in a specific *\maps* subdirectory). By zipping this *\Models\MyFirstCormasModel* subdirectory, you create an archive containing everything needed to install and run your model on another computer. The reverse process consists in unzipping this archive under the *\Models* subdirectory.

To build the list of models proposed by the **Model -> Import** option from the cormas main menu, Cormas is looking for all the subdirectories attached to *\Models*. When you want to load a new or updated model into Cormas, you can use this option.

4 CORMAS VISUALISATION'S TOOLS

4.1 SPATIAL INTERFACE

This interface is used to create a spatial context and to visualise the evolution of spatial entities and situated agents. To open the spatial interface click on the following icon .



The spatial interface is opened.



The configuration of the spatial grid can be done through its menu. This menu is composed of three sub-menus

4.1.1 THE TESSELATION SUBMENU



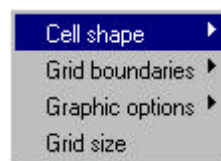
Cormas can represent the space as regular grids or irregular tessellation. Regular grids will be created automatically. Irregular grids will require special programs to generate or load polygons. Depending whether the “**Regular**” or “**Irregular**” option is selected within this submenu, the main menu is adjusted.

4.1.2 TOPOLOGY

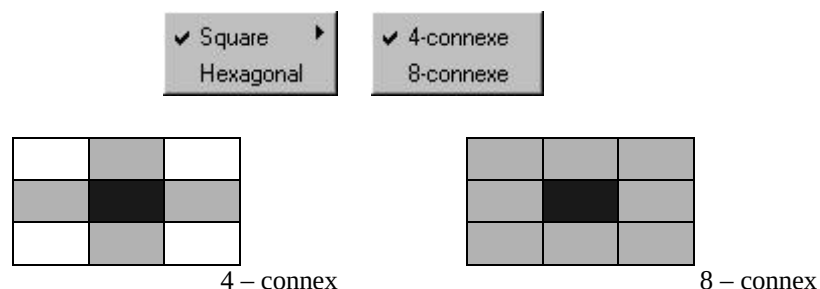
In case of irregular tessellation two options are proposed. One for the generation of the elementary spatial entities using the Voronoi algorithm, and another one to load polygons exported by Mapinfo software.



In case of regular grids four sets of options are proposed.



“**Cell shape**” allows to choose among rectangular or hexagonal cells. The hexagonal cell has six neighbours. The connexity of rectangular cells has to be chosen .



“Grid boundaries”

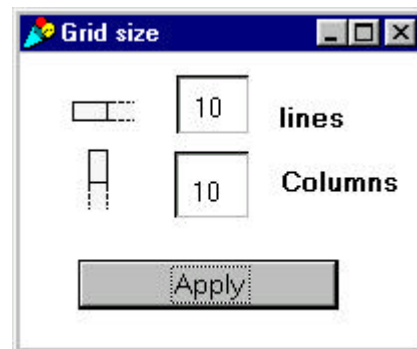
As the Cormas spatial grid can represent only a portion of the studied real environment, users can choose between two options for the grid boundaries:



- **“Toroidal”** With this type of boundary, all the cells of the grid have the same number of neighbouring cells, even those located at the edge of the grid. Phenomena that reach the edge of the grid can continue on the opposite side of the grid. One can visualise this type of space as an inner tube.
- **“Closed”** Cells at the edge of the grid have a number of neighbouring cells limited by borders.

“Define the grid size”

You can choose the number of lines and columns of the grid.

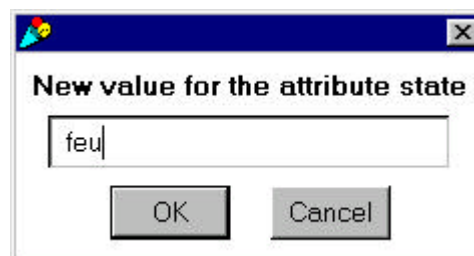


Validating will reinitialise the grid.

4.1.3 TOOLS

4.1.3.1 Click to change attribute... versus Click to inspect

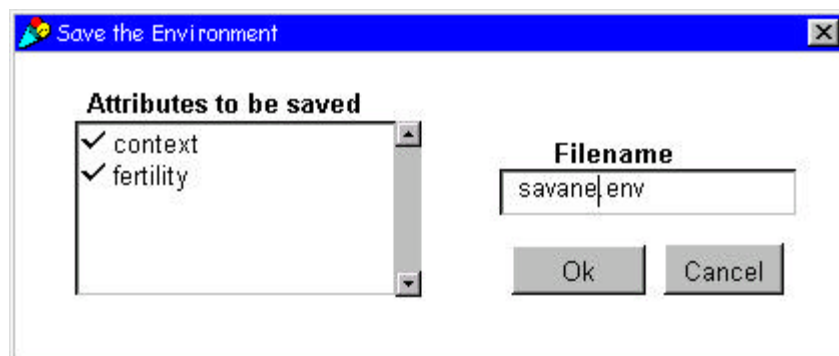
There are two alternative functions assigned to the left-click button of the mouse. Either it is used to change manually values of attributes, either it is used to inspect the cell. In the first case, a submenu made of the attributes of the active (last selection from the point-of-view contextual menu) spatial entity is automatically created. A dialog box requests the value to assign for the selected attribute.



Click on the **“OK”** button to validate. To assign this value to a particular cell, click on it. The cell will then update its attribute with this value. Sliding the cursor on several cells while clicking will change the state of these cells. During this operation, the cursor has changed. To turn back to the inspect function, simply select **Tools -> Click to inspect**.

4.1.3.2 Saving and loading the whole spatial grid

To save the topology and the values of the attributes of the active spatial entity at the end of a simulation for example, click on “**Save the environment**”.



A window appears, requesting for a selection of the attributes to be saved and for a filename. Such a file is created in the maps subdirectory of the model. It takes a .env extension. The header of the *savane.env* file from the **LandDyna** model is given below to describe the format adopted for such files.

```
dimensions 32 50
cloture    closed
connexite  eight
delimiteur 0
attributs  context(Symbol) fertility(Symbol)
forest,notFertile
forest,notFertile
forest,notFertile
...
```

The 4 first lines record the topology of the grid, the fifth corresponds to the selection of attributes (their type is also indicated between brackets), and then the values of the attributes are given, one line by cell, from the top left to the bottom right cell.

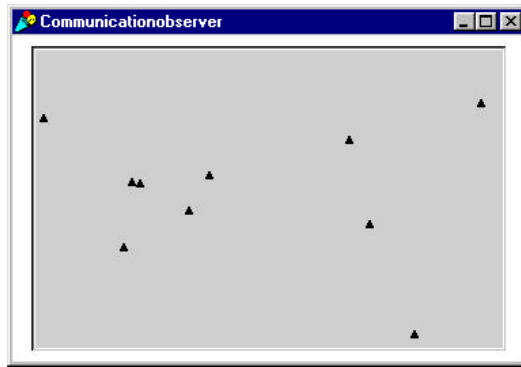
4.2 COMMUNICATION INTERFACE

To observe the agents' conversations, click on the icon

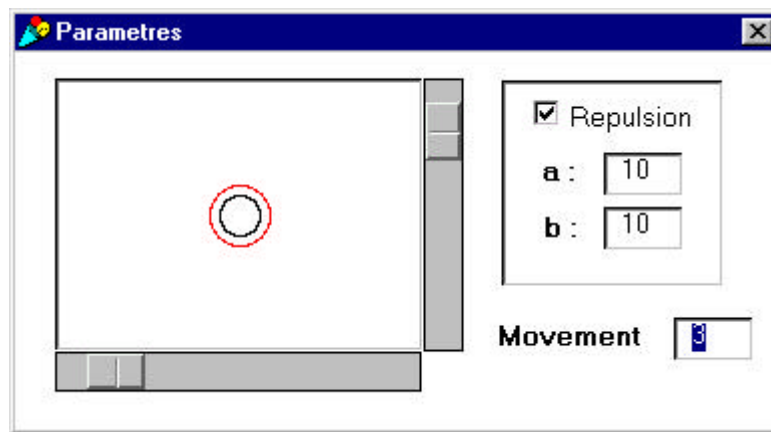


A window opens with triangles representing agents.

When the simulation begins a line will be displayed between two agents exchanging messages. Those agents who communicate attract themselves and those who do not communicate are repulsed. This is an algorithm proposed to visualise the graphs and isolate the sub groups.



In the contextual menu of that interface the action “**Parameters**” allows to set the speed of the agent's movement (“**Movement**”) and two radius, one for repulsion one for stabilisation.



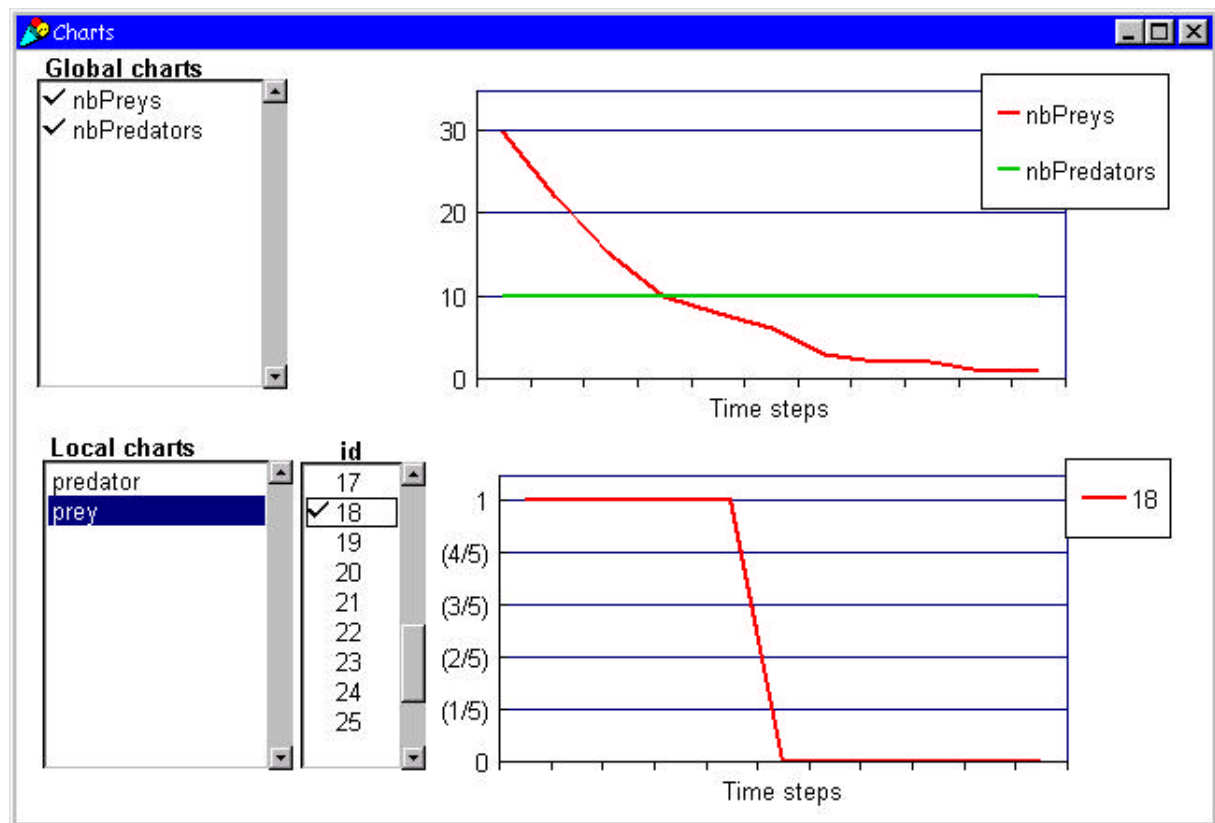
The repulsion radius (black) gives the minimal distance between an agent and another one. The area between repulsion and stabilisation radius (black) gives the zone where a communicating agent will stabilise .

4.3 CHARTS INTERFACE

To observe graphics created during the simulation, click on the corresponding icon from the "Visualisation tools" zone.



In the upper part of the opening interface, just select one or several pre-defined global charts (see section 2.3.3). In the lower part of this window; the list of pre-defined local (individual) charts allows only single selection. Once a local chart name is selected, a list made of all the id of the instances of the corresponding entity that have exist during the simulation is appearing. From this list of ids, multiple selection is allowed.



The results of a run of the **Pursuit** model are shown above. In this simple model, the predators do not have any chance to reproduce or to die, whereas the preys may disappear (it has occurred at time-step 4 for the prey #18).

5 RUNNING SIMULATIONS WITH CORMAS

The simulation zone at the bottom of Cormas main window is for driving simulations. The same set of buttons is available in the Cormas floating menu, which is a reduction of Cormas main window with only simulation tools. You can open it by clicking on the **>-<** button in the "Simulation" zone.

Buttons in the simulation zone activate methods programmed in the "Control the evolution" zone ("**Prepare and Schedule**" button). Clicking on "**Initialise...**" button opens a window for selecting initialisation and control methods. Validating will activate the initialisation method.

To activate the control method of the model, one can choose between using the step-by-step mode ("**Step**" button), or entering the number of time steps (input field "**Final time**") and click the "**Run**" button. In the "**Time**" box the number of elapsed time steps will appear.

The "**Run N times**" button can only be used if the selected control method calls two arguments, the second one corresponding to "**N**".